

Programación Orientada a Objetos

Práctica: Manejo de Excepciones en Java

Objetivo

Implementar un sistema orientado a objetos que utilice:

- Excepciones personalizadas
- try / catch / finally
- throw y throws
- Herencia
- Interfaces
- Encapsulamiento
- Colecciones

Duración estimada: 90–120 minutos.

Contexto

Una universidad desea implementar un sistema simple para gestionar los préstamos de materiales de su biblioteca.

El sistema permitirá:

- Registrar usuarios
- Registrar materiales
- Realizar préstamos
- Realizar devoluciones
- Registrar errores en un archivo de log

Durante las operaciones pueden ocurrir diversas situaciones excepcionales que deben manejarse adecuadamente.

Diagrama UML

Los estudiantes deben implementar el siguiente modelo.

Interfaz

Prestable

```
+ prestar() throws MaterialNoDisponibleException
+ devolver()
+ estaPrestado(): boolean
```

Clase abstracta

Material

```
- id : String
- titulo : String
- prestado : boolean

+ getId()
+ getTitulo()
+ estaPrestado()
+ setPrestado(boolean)
```

Clases concretas

Libro

```
- autor : String
- isbn : String
```

Implementa la interfaz **Prestable**

Revista

```
- numero : int
```

Implementa la interfaz **Prestable**

Clase Usuario

```
- id : int
- nombre : String
- prestamos : List<Material>

+ prestarMaterial(Material)
+ devolverMaterial(Material)
+ cantidadPrestamos()
```

Restricción:

Un usuario puede tener máximo **3 préstamos activos**.

Clase Biblioteca

```
- materiales : List<Material>
- usuarios : List<Usuario>

+ registrarMaterial(Material)
+ registrarUsuario(Usuario)

+ buscarMaterial(String id)
+ buscarUsuario(int id)

+ prestarMaterial(int usuarioId, String materialId)
+ devolverMaterial(int usuarioId, String materialId)
```

Jerarquía de Excepciones

Crear la siguiente jerarquía:

```
BibliotecaException
|
+-- MaterialNoDisponibleException
+-- MaterialNoEncontradoException
+-- UsuarioNoEncontradoException
+-- MaximoPrestamosException
```

Todas deben heredar de:

Exception

Uso de throw

Cuando ocurra una situación inválida se debe lanzar una excepción.

Ejemplo:

```
if(material.estaPrestado()){  
    throw new MaterialNoDisponibleException(  
        "El material ya está prestado"  
    );  
}
```

Uso de throws

Los métodos que puedan producir errores deben declararlo.

Ejemplo:

```
public void prestarMaterial(int usuarioId, String materialId)  
    throws UsuarioNoEncontradoException,  
           MaterialNoEncontradoException,  
           MaterialNoDisponibleException,  
           MaximoPrestamosException
```

Clase de Registro de Errores

Crear una clase:

RegistroErrores

Responsabilidad: registrar errores en un archivo.

Método:

```
registrar(Exception e)
```

Debe escribir en un archivo llamado:

```
errores.log
```

Formato sugerido:

```
[fecha] TipoDeExcepcion - mensaje
```

Programa Principal

Crear una clase:

Main

Debe simular el funcionamiento del sistema.

Ejemplo de flujo:

1. Crear la biblioteca
2. Registrar varios materiales
3. Registrar varios usuarios
4. Realizar préstamos
5. Realizar devoluciones

Las operaciones deben ejecutarse usando:

`try / catch / finally`

Ejemplo conceptual:

```
try {  
  
    biblioteca.prestarMaterial(1, "L001");  
  
} catch (BibliotecaException e) {  
  
    System.out.println(e.getMessage());  
    RegistroErrores.registrar(e);  
  
} finally {  
  
    System.out.println("Operacion finalizada.");  
  
}
```

—

Escenarios que deben probar

El programa debe demostrar al menos estos casos:

1. Préstamo exitoso
2. Intentar prestar un material ya prestado
3. Intentar prestar un material inexistente
4. Intentar prestar con usuario inexistente
5. Superar el máximo de préstamos permitidos

—

Desafío adicional

Si terminan antes, implementar:

- Persistencia simple: guardar los materiales en un archivo.
- Un menú interactivo en consola:

1. Registrar usuario
2. Registrar material
3. Prestar material
4. Devolver material
5. Salir

- Crear una excepción adicional:

`OperacionBibliotecaException`

que encapsule otras excepciones.

—

Criterios de evaluación

- Correcto uso de excepciones
- Diseño orientado a objetos
- Uso adecuado de herencia e interfaces
- Manejo de errores
- Claridad del código